Street Level Companion App: Build Spec

Nov 10, 2026 · @Stephen

Summary

A web app for running Street Level at one table. Each player opens a link on a phone, picks a character and edits their own sheet. The GM sees every sheet plus the campaign tracker, the hidden case file, a shared dice log, an encounter simulator and a rules helper.

Who	What they get
Player	Their character sheet, live and editable: Harm, Stress, Drive, gear, powers, clocks, a dice roller and the table's roll log
GM (Stephen)	Everything players have for every character, plus the Night tracker, case file, fight tracker, encounter simulator, rules helper, and export and import
Claude	Changes rules and balance data through the GitHub repo. Updates a campaign after a voice session through an import file

Not in version 1: remote play (voice, video, maps), accounts with passwords, rules the app enforces against the GM's wishes, simulated fights with more than one named villain, and native phone apps.

This spec is written for Claude Code to build from, one milestone at a time. The rules themselves live in `rules/rules.md`, and the spec never restates them: where a screen needs a rule, it points to the rules file.

Decisions and principles

These were settled with Stephen on Nov 10 and are not open for the build to change.

Area	Decision
Hosting	A GitHub repo. The site is static and deploys to GitHub Pages from an Action

Area	Decision
Shared data	A new Supabase project: Postgres, Realtime, Auth and Edge Functions
Sign-in	No accounts. Each device signs in anonymously. A join link makes it a player, a passcode makes it the GM
Players	Same table, phones and tablets. They join by link or QR code and pick a character
Dice	Every roll can be rolled by the app or typed in from real dice. The app does the math either way
Rules helper	A GM-only question box answered by Gemini, with the key held in an Edge Function
Simulator	The existing Python simulator, run in the browser against the campaign's real characters
Claude	Rules and balance changes come as repo commits. Campaign changes come as import files

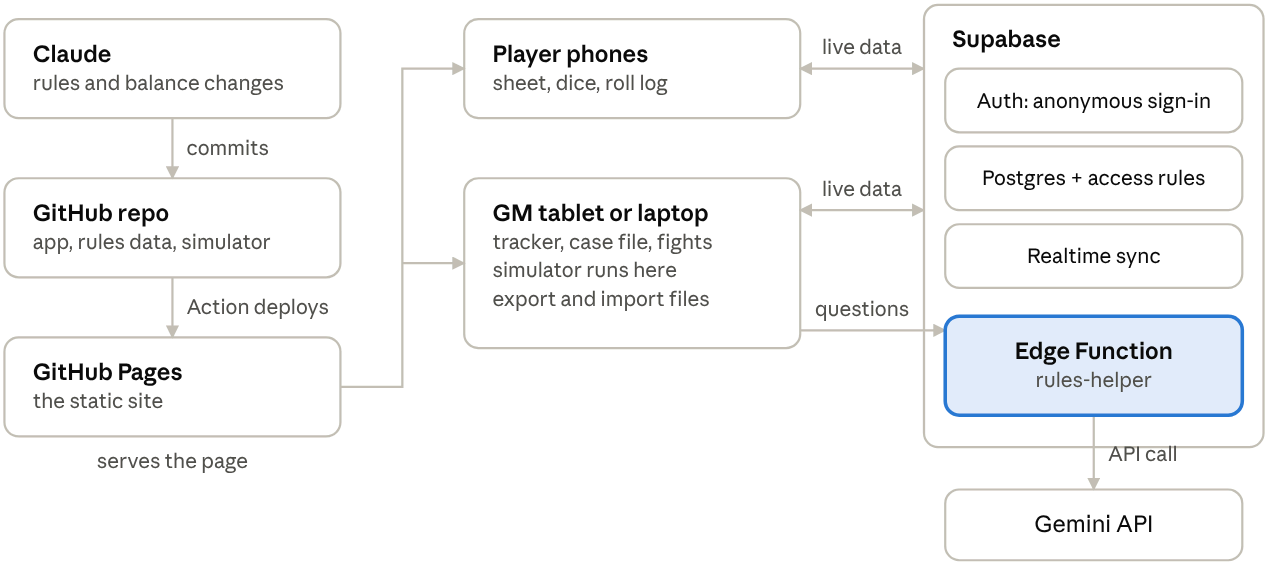
Principles for every screen:

1. **Track, don't adjudicate.** The app does arithmetic and bookkeeping and shows the rule text. Every tracked value can be changed by hand. The rules are still changing, so a warning is fine and a hard block is not.
2. **One source of rules data.** Sheets, the character builder, the dice roller and the simulator all read the same files in `rules/data/`. A balance change is made once.
3. **Secrets never reach a player's device.** The database enforces this. Hiding something in the page is not enough.
4. **Phone first.** Every player screen works one-handed on a 380 px wide screen. The GM screens target a tablet or laptop.
5. **One simulator.** `sim/street_level_sim.py` stays a single Python file that runs in the browser and from the command line. Claude runs the same file in its own workspace for balance work.
6. **Small writes.** A change writes only the field that changed, so two people editing the same character don't overwrite each other.

Architecture and repo layout

The page is static. All shared state lives in Supabase, the simulator runs inside the GM's browser, and the only server code is one small Edge Function for the rules helper.

One static page, one database, one small server function



architecture · who loads what and who talks to what

Phones and the GM's device load the same page and sync through Supabase. The rules helper (highlighted) is the only code that runs on a server.

Part	Choice
Web app	Vite, React, TypeScript. One app with player and GM routes
Data access	`supabase-js` : anonymous Auth, Postgres with row-level security, Realtime subscriptions
Simulator	Pyodide in a Web Worker, loaded only when the GM opens the Encounter lab
Rules helper	Supabase Edge Function `rules-helper` that calls the Gemini API
Deploy	GitHub Actions: test, build, publish to Pages. Database changes are SQL migration files

Part	Choice
Tests	Vitest for the rules engine, pytest for the simulator, a script that checks the access rules

```
street-level-app/
  SPEC.md
  rules/
    rules.md                the rules text (export of the rules doc); also
fed to the helper
    data/                   machine-readable rules, see "Rules data"
  sim/
    street_level_sim.py     one file: command line plus
run_encounter(config)
    tests/
  web/
    src/
      engine/               derived stats, roll math, Harm math, sheet-to-
simulator mapping
      data/                 Supabase client, queries, realtime hooks
      screens/player/
      screens/gm/
      sim/                  the worker that loads Pyodide and the simulator
file
      components/
  supabase/
    migrations/             tables, access policies, RPC functions
    functions/rules-helper/
    seed/                   demo campaign: the five sample heroes and the
Butcher case
  .github/workflows/        test.yml, deploy.yml
```

The build copies `rules/` and `sim/street_level_sim.py` into the site's static files. The page, the simulator worker and the helper function then all read the same published versions.

Rules data

Every number and rule text the app shows comes from JSON files in `rules/data/`. Claude Code generates them once from `rules/rules.md`, and after that they are edited by hand or by Claude's commits.

File	Holds
`core.json`	`rules_version`, defense base (12), Impact bands, Harm box names, Stress stages, Drive prices and starting Drive (2), slot costs, clock sizes, Dawn options, villain turn and gang size scaling
`archetypes.json`	The 5 archetypes: Drive trigger, feature, Quick Actions, Signature
`traits.json`	The 8 Traits: Drive trigger, Steady, Fraying, hook, Breaking
`flairs.json`	The 9 Flairs
`gear.json`	Every gear item: slots, text, uses per fight
`kit.json`	The 7 Kit items
`powers.json`	Street and strange powers: tier, the 3 abilities, always-on flag, flaw rules
`villains.json`	Stat blocks (the Butcher, Voss, the Saints, a lieutenant template) and the four roles
`tags.json`	Zone tags and their effects

Entry shape. Each entry has a stable `id`, a `name`, the `text` shown on sheets, and an optional `sim` object that tells the simulator what the entry does:

```
{"id": "kevlar", "name": "Kevlar vest", "slots": 1,
 "text": "Toughness 1 against guns and blades. Doesn't stack with other
Toughness.",
 "sim": {"armor_guns": 1}}

{"id": "weighted-gloves", "name": "Weighted gloves", "slots": 1,
 "text": "Light melee: +1 Harm on Impact 2 or better. Works with Strikes and
Jabs.",
 "sim": {"weapon": "light"}}

{"id": "weather-control", "name": "Weather Control", "tier": "strange",
"slots": 2,
 "abilities": ["...", "...", "..."],
 "sim": null}
```

`"sim": null` means the simulator can't model the entry. The Encounter lab lists those under "Not simulated" so the GM knows what the number leaves out.

Allowed `sim` fields (they match the simulator's hero fields):

Field	Values
`weapon`	`unarmed`, `light`, `heavy`, `pistol`, `knife`
`armor_guns`, `armor_all`, `tough_blunt`	0 or 1
`guard_bonus`, `boxes_bonus`, `medkit`	whole numbers
`power`	`Toughness`, `Shadow Manipulation`, `Mighty`, `Invulnerability`, `Healing Factor`
`strange`	true for strange-tier powers (first use costs 1 Stress)
`flair`	`hard_to_kill`, `opportunist`
`kit`	`taser`, `flashbang`, `smoke`, `armor`

The rules engine (`web/src/engine/`) is a set of pure functions with no network calls:

- `defenses(sheet)`, `toughness(sheet, attackKind)`, `harmBoxes(sheet)`, `slotsUsed(sheet)`
- `resolveRoll({dice, bonus, edge, adv, defense})` returns kept dice, total, margin and Impact
- `harm({impact, weapon, targetToughness, gun, fromMook, jab})` returns Harm with the steps shown
- `simHero(sheet)` turns a character sheet into the simulator's hero record, using the `sim` fields

Versioning. `core.json` carries `rules_version` (a date string). Each roll in the log stores the version it was made under, and a campaign shows a notice when the published version is newer than the one it last loaded.

Data model

Twelve tables, all keyed by `campaign_id`, split into what the whole table can see and what only the GM can see. Every table has `id`, `campaign_id` and `updated_at` unless noted.

Shared with the table

Table	Columns that matter
`campaigns`	`name`, `join_code` (short, goes in the player link), `gm_code_hash`, `rules_version`, `night`, `phase` (`patrol`, `crack`, `confrontation`, `dawn`), `settings` (players can build characters, players can see other sheets)
`members`	`user_id` (the anonymous Auth user), `role` (`player` or `gm`), `character_id`, `display_name`
`characters`	`name`, `civilian`, `archetype_id`, `trait_id`, `stats` (`force`, `flow`, `focus`, `presence`), `flair_ids`, `loadout` (list of gear, powers and Kit picks: `kind`, `id`, `ability_level`, `flaw`, `custom_text`), `harm`, `stress`, `drive`, `dying`, `hidden`, `home_clock`, `fight_state` (per-fight flags: strange power used, Second Wind used, Kit used, item uses), `notes`, `is_npc`, `locked`
`clocks`	`name`, `kind` (`scheme`, `heat`, `goal`, `endgame`, `custom`), `size`, `filled`, `visible_to_players`, `sort`
`clues`	Found clues only: `text`, `found_by`, `personal_for`, `is_arc`, `night`
`scene`	One row per campaign: `title`, `text`, `round`, `zones` (`id`, `name`, `tags`, `links`), `positions` (who is in which zone), `public_enemies` (what players may see of the enemies)
`rolls`	`at`, `actor_name`, `character_id`, `label`, `stat`, `bonus`, `edge`, `adv`, `defense`, `dice`, `kept`, `total`, `margin`, `impact`, `mode` (`app` or `manual`), `note`, `rules_version`

GM only

Table	Columns that matter
`gm_secrets`	One row per campaign: `case` (title, mode, Complexity, culprit, the three facets, cast, leads with ratings, the full clue list, Scheme steps, personal clues, confrontation), `arc` (Mastermind, Tools, calling card, arc clues), `notes`
`enemies`	`name`, `kind` (`mooks`, `lieutenant`, `villain`), `stats`, `defenses`, `toughness`, `harm`, `harm_max`, `stress`, `count`, `count_max`, `drive`, `zone`, `line` (one-line summary), `source_id` (the villain library entry)
`encounters`	`name`, `config`, `last_result`
`imports`	`at`, `note`, `patch`, `before` (for undo), `applied`
`helper_log`	`at`, `question`, `answer`, `model`

Notes for the build

- Unfound clues live only inside `gm_secrets.case`. Revealing one inserts a row into `clues`.
- Players never read `enemies`. When the GM changes an enemy, the GM's page rewrites `scene.public_enemies` with the safe summary: name, zone, mooks standing, and a rough state (fresh, hurt, badly hurt).
- Realtime is on for `campaigns`, `characters`, `clocks`, `clues`, `scene` and `rolls`.
- Tracker changes (`harm`, `stress`, `drive`, `filled`) update that one column. Changes inside `fight_state` and `loadout` go through small RPC functions that patch one key.
- Seed data: one demo campaign with the five sample heroes from `rules.md` and the Butcher case from the current GM Table. Claude will supply the case as a JSON file.

Access, roles and secrets

Nobody types a username or password. Each device signs in anonymously the first time it loads the page, and the database decides what that device may read and write.

How a device gets its role

1. On first load the page calls Supabase anonymous sign-in. The device now has a user id, kept in the browser's storage.
2. **Player:** the link `.../#/join/<join_code>` calls the RPC `join_campaign(join_code)`, which adds a `members` row with role `player`. Picking a character calls `claim_character(character_id)`.
3. **GM:** the page `.../#/gm` asks for the campaign and its passcode and calls `claim_gm(campaign_id, passcode)`. The function compares the passcode to `gm_code_hash` and sets the role to `gm`. The passcode is chosen when the campaign is created.

Who can do what (row-level security on every table)

Data	Player	GM
`campaigns`, `clocks` marked visible, `clues`, `scene`	Read	Read and write
`characters`	Read all (or only their own, per campaign setting). Write the one they claimed	Read and write all
`rolls`	Read all. Add rolls for their own character	Read, add, delete
`gm_secrets`, `enemies`, `encounters`, `imports`, `helper_log`, hidden clocks	Nothing	Read and write

Rules for the build

- Anonymous users count as signed-in users in Supabase, so a policy that only checks "is signed in" lets anyone in. Every policy must check for a `members` row in that campaign, and GM policies must check `role = 'gm'`.
- The three RPC functions are `security definer` and are the only way to create or change a `members` row.
- The Supabase project URL and anon key are public by design and sit in the page. The service role key and the Gemini key exist only as Edge Function secrets. Neither may appear in the repo, the page, or a chat.
- If a browser's storage is cleared, the device rejoins with the link. Two devices may claim the same character.

- Ship a script (`supabase/tests/access.test.ts`) that signs in as a stranger, a player and a GM and proves each row of the table above.

References to check against at build time: Supabase anonymous sign-ins.

Player screens

A player has four screens: Join, Sheet, Rolls and Builder. The Sheet is where they spend the session, so it is one scrolling page with the three trackers pinned at the top.

Join. Open the link or scan the QR code. The page lists the campaign's characters, unclaimed ones first. Tap one to claim it. If the campaign allows it, a "Make a new character" button opens the Builder.

Sheet

Block	Contents
Pinned header	Name, archetype, Trait. Harm boxes by name (tap to fill or clear). Stress boxes with the stage label (Steady, Fraying, Breaking) and the Trait's line for that stage. Drive pips (max 6)
Stats and defenses	The four stats, each with a Roll button. Guard, Wits and Poise, worked out by the engine. Toughness and what it covers
Actions	The Full and Quick Action lists, the archetype's Drive trigger and feature, and the Signature with a "Spend 3 Drive" button. Buttons for Second Wind (3 Drive), Strain (1 Stress) and Edge (1 Drive)
Loadout	Each gear item and power with its text. Powers show which abilities are unlocked, a Push button (1 Stress), the flaw, and the strange-power first-use Stress toggle. Items with limited uses have counters. Specialists pick their 3 Kit items here at the start of a Night
Clocks and clues	The hero's Home clock (editable). The team's Scheme and Heat clocks and the list of found clues (read only)

Block	Contents
Notes	Free text, visible to that player and the GM

Two buttons do bookkeeping:

- **New fight** clears the per-fight flags: strange power used, Second Wind used, Kit and item uses.
- **Dawn** opens a dialog to pick two of the five Dawn options and applies them as written in `rules.md`: clear Stress, clear Hurt and Wounded, tick Home if Show Up was not picked, lower Heat, or ask the GM for a clue. It then drops Drive above 3 to 3.

Every change the buttons make can be undone by tapping the tracker.

Rolls. The dice roller and the table's roll log. See "Dice roller and roll log".

Builder. Name, civilian identity, archetype, Trait, a stat array assigned to the four stats, one Flair, and the loadout. A counter shows slots used out of 3: gear and street powers cost 1, a strange power costs 2 and needs a flaw, and only one strange power is allowed at creation. A Specialist's Kit sits outside the slots. Breaking a rule shows a warning and does not block saving. The same screen handles Advancement when a case closes.

GM screens

The GM has seven screens. The rules helper box is docked on all of them.

Screen	What it does
Table (home)	A compact, editable card per hero with the same trackers as the player sheet. The Night number and phase, the Scheme and Heat clocks, the join QR code, and the roll log

Screen	What it does
Night	Steps through Patrol, Crack the Case, Confrontation and Dawn. A "Tick Scheme" button asks who takes the Stress (the hero whose roll caused it, or everyone) and applies it. A Crack the Case helper counts the clues in the theory (personal clues count double), shows the target, and records the result. A Dawn view shows everyone's two picks. "Start Night" raises any hero below 2 Drive to 2
Case file (hidden)	The culprit, the three facets, the cast, leads with their rating and defense, the full clue list with a Reveal button, the Scheme steps, and the arc sheet (Mastermind, Tools, calling card, Endgame clock, arc clues). Revealing a personal clue offers to tick that hero's Home clock
Fight	A zone board (boxes with tags; tap a token, then a zone, to move it). An enemy panel: add from the villain library or make one, then track Harm, Stress, Drive and mooks standing. A round counter. A reminder of villain turns and gang size for the current party size. Goal clocks with an "End of round" button that ticks them last. "End fight" stands Down heroes who are not Dying back up at Wounded and clears per-fight flags
Encounter lab	The simulator. See "Encounter simulator"
Library	Read-only views of gear, powers, Flairs and villains from the rules data, plus the campaign's own custom villains
Admin	Create a campaign and set its passcode. Unclaim or lock characters. Toggle whether players can build characters and see each other's sheets. Export, import, and the session report

Later, not in the first pass: a read-only Table display for a TV or spare tablet that shows the zone board, public clocks and the roll log.

On every screen the GM can edit any number directly. Buttons like "Tick Scheme" and "End fight" are shortcuts for bookkeeping the GM could do by hand.

Dice roller and roll log

Every roll goes through one form, whether the app rolls the dice or the player types in what their real dice showed. The app then does the math, shows its working, and logs the roll for the whole table.

The form, in order

1. **Who.** A player's own character is filled in. The GM picks any hero or enemy.
2. **What.** A stat (bonus filled in from the sheet) or a typed bonus, plus a short label such as "Haymaker on the Butcher".
3. **Modifiers.** Edge (free, or "spend 1 Drive"), Advantage or Disadvantage, and Jab (-2).
4. **Against.** One of: an enemy's Guard, Wits or Poise (GM only, since players can't see enemy numbers), a hero's defense, an obstacle rating from -1 to +5 (the app shows 12 + rating), or a typed number the GM calls out.
5. **Dice.** "Roll for me", or "I rolled" with a box per die: two dice, or three with Advantage or Disadvantage. Each must be 1 to 10.

The result card shows the dice, the kept dice, the total, the margin, the Impact, and the line from the Impact table. For an attack it adds a Harm line with each step shown: Impact, weapon bonus, Toughness, the 1-Harm minimum, the mook cap. When the Impact is 1 or 2 it offers "Upgrade (2 Drive)". The GM gets an Apply button that puts the Harm or Stress on the target.

Rules the engine must get right (each is a test case; the text is in `rules.md`)

- Advantage and Disadvantage cancel. Only one Edge applies. Edge and Advantage stack.
- Impact tops out at 3. Weapon and power bonuses add Harm, never Impact.
- A Jab that hits is always Impact 1, so bonuses that need Impact 2 never apply to it.
- Guns add 1 Harm on Impact 2 or better and ignore Toughness. Mighty ignores Toughness.
- A hit always does at least 1 Harm. Mook attacks cap at 2 Harm, applied last.

The log. Rolls appear for everyone within a couple of seconds, newest first, with typed rolls marked "manual". The GM can delete a roll. An export button writes the log as CSV in the column order of the playtest logs (`actor, action, stat, bonus, adv, defense,`

`dice, kept, total, margin, result, note`), so Claude can audit a session.

App dice use `crypto.getRandomValues` with rejection sampling, so each face is equally likely.

Encounter simulator

The GM builds an encounter, presses Run, and gets a difficulty label for the campaign's real characters. The engine is the existing `street_level_sim.py`, running in the browser.

Changes to the simulator file

Today the file has five hardcoded sample heroes (`make_heroes`) and four named scenarios, and it is driven by command-line flags. It needs one new entry point and nothing else about its fight logic should change:

- Add `run_encounter(config: dict) -> dict`. It builds heroes and enemies from the config, runs the fights, and returns a plain dict. No printing and no `argparse` inside it.
- Heroes come from `config["heroes"]`, each a record with the `Hero` fields. Enemies come from `config["villain"]` (optional, the `Boss` fields) and `config["mook_groups"]` (a list with the `Mooks` fields).
- Each hero may carry starting `harm`, `stress` and `drive`, so a fight can be run with the party as they are right now.
- `config["rules"]` overrides fields of the `Rules` dataclass. Missing fields keep their defaults.
- Keep the command line working: the existing flags, plus `--config encounter.json`.

Config (built by `simHero(sheet)` and the Encounter lab form)

```
{
  "runs": 1000, "seed": 7,
  "heroes": [
    {"key": "rook", "name": "Rook", "arch": "Brawler", "trait": "Spite",
     "force": 4, "flow": 2, "focus": -1, "presence": 1, "boxes": 5,
     "weapon": "light", "armor_guns": 1, "medkit": 1, "flairs":
["hard_to_kill"],
     "power": "", "strange": false, "kit": [],
     "harm": 0, "stress": 0, "drive": 2,
     "not_simulated": []}
  ],
  "villain": {"name": "The Butcher", "force": 4, "flow": 0, "focus": 1,
"presence": 2,
              "harm_max": 10, "toughness_v": 1, "drive": 2, "weapon":
"unarmed"},
  "mook_groups": [{"name": "Saints", "count": 3, "attack": 2, "guard": 13,
"gunner": true}],
  "rules": {"dark": false}
}
```

Result

```
{
  "win_pct": 97.6, "rounds": 3.8, "any_down_pct": 21.9, "dying_pct": 0.0,
  "heroes": {"rook": {"down_pct": 7.0, "harm": 2.16, "hit_pct": 73, "stress":
0.37,
                      "fraying_pct": 1.4, "break_pct": 0.1, "sigs": 1.22,
                      "drive_earned": 3.75, "drive_spent": 3.85}},
  "not_simulated": ["Nightjar: grapple line"],
  "runs": 1000, "seed": 7, "sim_version": "2026-11-10"
}
```

The Encounter lab screen

- **Party:** all player characters by default, with checkboxes. A switch chooses "fresh" or "as they are now" (current Harm, Stress and Drive).
- **Enemies:** at most one named villain or lieutenant, from the library or custom, plus any number of mook groups. The form suggests the gang size from the scaling rule for the party size.
- **Options:** dark zone, number of runs (default 1,000).

- **Result:** a label, the four headline numbers, a bar per hero for their chance of going Down, and the "Not simulated" list.
- Encounters can be saved, renamed and rerun.

Difficulty labels are starting bands, to be adjusted after real sessions:

Simulated win rate	Label
97% or more	Walkover
85 to 97%	Fair
65 to 85%	Hard
Under 65%	Deadly

Add the flag "Someone probably goes Down" when `any_down_pct` is 50 or more. Under every result, print: "The simulator plays simple tactics and ignores zones and goals. A real table usually does better."

How it runs. A Web Worker loads Pyodide and the published simulator file the first time the Encounter lab opens, then keeps them in memory. The page shows progress while it loads and while fights run.

Known limits to show on the screen: one named enemy per fight, no zones or goal clocks, fixed tactics, five powers modeled (Toughness, Shadow Manipulation, Mighty, Invulnerability, Healing Factor) and four Kit items (Taser, Flashbang, Smoke, Armor).

Checks

- pytest: Rook, Nightjar and the Ledger against the Butcher and one Saints group, 2,000 runs, wins between 95 and 99% (the command line gave 97.6% on Nov 10).
- The browser and the command line agree within 2 points for the same config.
- Speed is unmeasured in a browser. Target: 1,000 fights with four heroes in under 10 seconds on a mid-range laptop. If it misses, lower the default to 500 runs. Rewriting the simulator in TypeScript is a last resort and needs Stephen's sign-off.

Rules helper box

A GM-only text box that answers "how do I run this?" questions from the rules text and the current party, using Gemini. Typical questions: "sneak past guards into a building", "save people from a burning building".

Flow

1. The GM types a question. A checkbox adds the current scene and enemies. A second checkbox, off by default, adds the hidden case file.
2. The page calls the Edge Function `rules-helper` with the campaign id, the question and the two checkboxes.
3. The function confirms the caller is that campaign's GM. It loads `rules.md` from the published site (kept in memory, reloaded when `rules_version` changes) and builds a party summary from `characters`: name, archetype, stats, defenses, loadout, current Harm, Stress and Drive.
4. It calls the Gemini API and returns the answer. The question and answer are saved to `helper_log`.

What the answer must contain (written into the system instruction)

- Use only the supplied rules. If the rules don't cover it, say so, then offer one suggestion labeled "Ruling".
- Name the roll: who rolls, which stat, and the defense as a number. Use the real heroes: "Nightjar rolls Flow +4 against 13".
- Say what a Miss and each Impact level mean in this situation.
- For Impact 1, offer two costs that fit the scene.
- Mention any clock, Stress or Drive effect.
- Stay under 150 words unless asked for more. Plain text, no headings.

Settings and limits

Setting	Value
Model	Set by the `GEMINI_MODEL` secret. Pick a fast, low-cost model from Google's current list at build time
Key	The `GEMINI_API_KEY` secret on the Edge Function. Nowhere else
Daily cap	60 questions per campaign per day, set by a secret
Question length	500 characters
Timeout	20 seconds, then a plain error message and a Retry button

Fallback. A "Copy question with context" button puts the question, the party summary and a link to the rules on the clipboard, for pasting into any chat.

Checks. The two example questions above and "Rook wants to throw a dumpster at the Butcher" each return a named roll, a defense number and two costs. A request from a player account is refused. The GM is the authority: the box says "Check the rule if this looks wrong" under every answer.

How Claude updates the app

Claude can commit to the GitHub repo but cannot reach Supabase from its workspace. So rules changes travel through the repo, and campaign changes travel through files that Stephen exports and imports.

What changes	How it travels	Built in
Rules text, rules data, the simulator	Claude commits to `rules/` and `sim/`. The site redeploys and campaigns show a "rules updated" notice	M1
Campaign state after a session	Stephen exports the campaign and attaches it. Claude returns a patch file. Stephen imports it	M3
Session evidence for balance work	A session report: roll log CSV, state at the start and end of the Night, helper questions, GM notes	M7

Export. One JSON file with every table for the campaign, the hidden ones included, plus `rules_version` and the export time. GM only.

Import. Accepts a patch as a file or as pasted text:

```
{
  "format": "street-level-patch/1",
  "campaign": "<campaign id>",
  "note": "After the Nov 14 voice session, Night 3",
  "ops": [
    {"table": "characters", "match": {"name": "Rook"}, "set": {"harm": 2,
"stress": 4, "drive": 1}},
    {"table": "clocks", "match": {"kind": "scheme"}, "set": {"filled": 3}},
    {"table": "clues", "insert": {"text": "A gas-soaked rag with a Tidewater
Seafood logo", "night": 3}},
    {"table": "gm_secrets", "merge": {"notes": "Marsh knows someone is
digging."}}
  ]
}
```

- Each op targets one table. `match` must find exactly one row, or the whole import is refused with the reason.
- Before anything is written, the screen shows a before and after line for every op.
- The import applies all ops or none, saves the old values in `imports.before`, and offers "Undo last import".

Voice sessions with Claude as GM. Until the Night and Fight screens exist (M4), the current GM Table artifact stays the tracker for those sessions. After that, Claude runs a session from an export and hands back a patch.

Later, if importing by hand gets tedious: Claude commits patch files to `campaign-updates/` and a GitHub Action posts them to an Edge Function `apply-patch` that checks a shared secret. Not in the first build.

Build order

Eight milestones, built in order. Each ends with a deploy and a check Stephen can do himself. Claude Code stops after each one and shows the check before starting the next.

#	Milestone	Delivers	Done when
M0	Foundation	Repo layout, the Vite app, the Pages deploy Action, all tables and access policies, anonymous sign-in, create campaign, join link, GM passcode	Two phones join the demo campaign. The access test script passes: a stranger reads nothing, a player can't read `gm_secrets` or `enemies`
M1	Rules data and engine	The JSON files generated from `rules.md`, the engine functions, their tests	The engine gives the five sample heroes these defenses (Guard / Wits / Poise): Bulwark 12 / 13 / 14, Rook 14 / 11 / 13, Nightjar 16 / 14 / 11, Patchwork 14 / 15 / 13, the Ledger 13 / 16 / 14
M2	Character sheets	The Builder, the player Sheet, live sync	A change on a phone shows on a second device within 2 seconds. A build using 4 slots shows a warning
M3	GM core	Table, Night and Case file screens, clocks, clues, Admin, export and import	The case file never appears in a player's network traffic. An export, a patch import and an undo all round-trip
M4	Dice and fights	The roller, the roll log, the Fight screen, enemies	20 rolls taken from the playtest logs give the same total, margin and Impact. App rolls and manual rolls both reach every device
M5	Simulator	`run_encounter`, the worker, the Encounter lab	The browser and the command line agree within 2 points. A hero with an unmodeled power appears under "Not simulated"
M6	Rules helper	The Edge Function and the docked box	The three test questions return a named roll, a defense number and two costs. A player's request is refused

#	Milestone	Delivers	Done when
M7	Polish	QR join, session report, offline badge and retry, the Table display	A phone that loses wifi for a minute shows the badge and catches up when it returns

Which model for which milestone. Opus for M0, M1, M3 and M5, where a mistake in the data model, the access rules or the simulator port is expensive. Sonnet for M2, M4, M6 and M7, which follow patterns the earlier milestones set.

How to start each one in Claude Code: "Read SPEC.md and rules/rules.md. Build milestone M<n>. Stop when its check passes and show me how to run the check."

Setup checklist for Stephen

Seven things only Stephen can do, because they need his accounts. The first four come before M0. The Gemini key can wait until M6.

- [ ] Create the GitHub repo (suggested name: `street-level-app`) and decide public or private (see "Open decisions").
- [ ] Create a new Supabase project. Note its project URL and anon key.
- [ ] In the Supabase project's Auth settings, turn on anonymous sign-ins.
- [ ] Put three files in the repo: this spec as `SPEC.md`, `street_level_rules.md` as `rules/rules.md`, and `street_level_sim.py` as `sim/street_level_sim.py`.
- [ ] Open Claude Code in the repo and start M0. It will ask for the project URL and anon key, and walk through linking the Supabase CLI.
- [ ] Before M6: create a Gemini API key in Google AI Studio and store it as the Edge Function secret `GEMINI_API_KEY`. Type it into the terminal yourself. Don't paste it into a chat or commit it.
- [ ] To let Claude commit rules changes from the design chat, attach the repo to that conversation.

Open decisions and risks

Six choices are still Stephen's. Each has a default the build uses if he doesn't say otherwise.

Decision	Default
Public or private repo? A public repo makes the rules text and simulator visible to anyone. As far as Claude knows, GitHub Pages on a free account needs a public repo, so a private repo would mean hosting the site somewhere else (Cloudflare Pages or Netlify). Check GitHub's current terms	Public
Can players build their own characters?	Yes, and the GM can lock a sheet
Can players see each other's sheets?	Yes, read only
What do players see of enemies?	Name, zone, mooks standing, and a rough state. No numbers
A hero ends a Night with 0 or 1 Drive. Do they start the next Night at 2?	Yes, raised to 2
One campaign or several?	Several, each with its own link and passcode

Risks

Risk	What the build does about it
The simulator is too slow or too big in a browser. Nobody has measured it	Load it only in the Encounter lab. Measure at M5. Fall back to 500 runs
The rules keep changing, and the JSON data drifts from `rules.md`	`rules_version` on everything. A test that fails when a gear, power or Flair named in the data is missing from `rules.md`
Table wifi drops	Trackers and dice keep working on the device and sync when the connection returns. An offline badge shows the state
Gemini gives a wrong ruling	Answers must name their roll and number so they are easy to check. The GM decides
A free Supabase project may pause after a period with no use. Check the current policy	Open the app the day before a session

Risk	What the build does about it
Someone outside the table finds the join link	They can only join as a player and see what players see. The GM can remove members and change the join code